

**MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

REPORT

Laboratory Work No. 7

Operating Systems

Three Small Programs: Processes, Redirection, and a Pipe

Student: Bianca-Andreea Tataroi
Group: FAF-242

Checked by: Reițman Patricia

Chișinău – 2026

Contents

1 Objective **2**

2 Laboratory Environment **2**

3 Part 1 – fork, exec, and wait **2**
3.1 Creating and Executing run.c 2
3.2 Replacing ls with date 3
3.3 Testing an Execution Failure 4
3.4 Part 1 – Observations 4

4 Part 2 – A Zombie, on Purpose **5**
4.1 Creating the zombie Program 5
4.2 Observing the Zombie in the Process Table 5
4.3 Inspecting the Process Table After the Delay 6
4.4 Part 2 – Observations 6

5 Part 3 – Redirection: ls -l to out.txt **6**
5.1 Understanding Standard File Descriptors 6
5.2 Creating and Testing redirect.c 6
5.3 Part 3 – Observations 7

6 Part 4 – A Pipe: ls /etc to wc -l **8**
6.1 Creating pipeline.c 8
6.2 Comparing the C Pipeline with the Shell 8
6.3 Leaving the Writing End Open 9
6.4 Restoring the Working Pipeline 9
6.5 Part 4 – Observations 10

7 Results and Discussion **10**

8 GitHub Repository **10**

9 Conclusion **11**

1 Objective

The objective of this laboratory work was to understand how Linux creates and executes processes, how standard output can be redirected to a file, and how two processes exchange data using a pipe.

The laboratory consisted of four mandatory experiments:

- creating child processes with `fork()`, running commands with `execlp()`, and collecting status with `waitpid()`;
- creating and observing a zombie process;
- redirecting standard output to `out.txt`;
- building a pipeline between `ls` and `wc -l` and investigating a deliberately unclosed pipe.

Four C files were created: `run.c`, `zombie.c`, `redirect.c`, and `pipeline.c`.

2 Laboratory Environment

The experiments were performed inside an Ubuntu Linux virtual machine using Bash, the Nano text editor, and GCC. The working directory was created using:

```
mkdir -p ~/os-lab7
cd ~/os-lab7
pwd
```

The working directory was:

```
/home/bianca/os-lab7
```

The compiler was checked with:

```
gcc --version | head -1
```

The installed compiler version was GCC 15.2.0. Programs were compiled with `-Wall -Wextra` to enable additional compiler warnings. A Markdown report, `report.md`, was also prepared as required by the assignment.

3 Part 1 – fork, exec, and wait

3.1 Creating and Executing run.c

The first program demonstrates how a shell launches `ls -l`: the parent uses `fork()` to create a child, the child executes `ls` with `execlp()`, and the parent calls `waitpid()` to collect the result.

The program was created, compiled, and executed with:

```
nano run.c
gcc -Wall -Wextra -o run run.c
./run
```

The central process-management instructions were:

```
pid_t pid = fork();
if (pid == 0) {
    execlp("ls", "ls", "-l", (char *)NULL);
    perror("exec");
    exit(127);
}
int status;
waitpid(pid, &status, 0);
```

The initial execution produced the following process IDs:

```
parent: my PID is 4683
child: my PID is 4684, my parent is 4683
parent: child 4684 ended with status 0
```

The child listed the current directory. Status 0 meant that the `ls` command finished successfully.

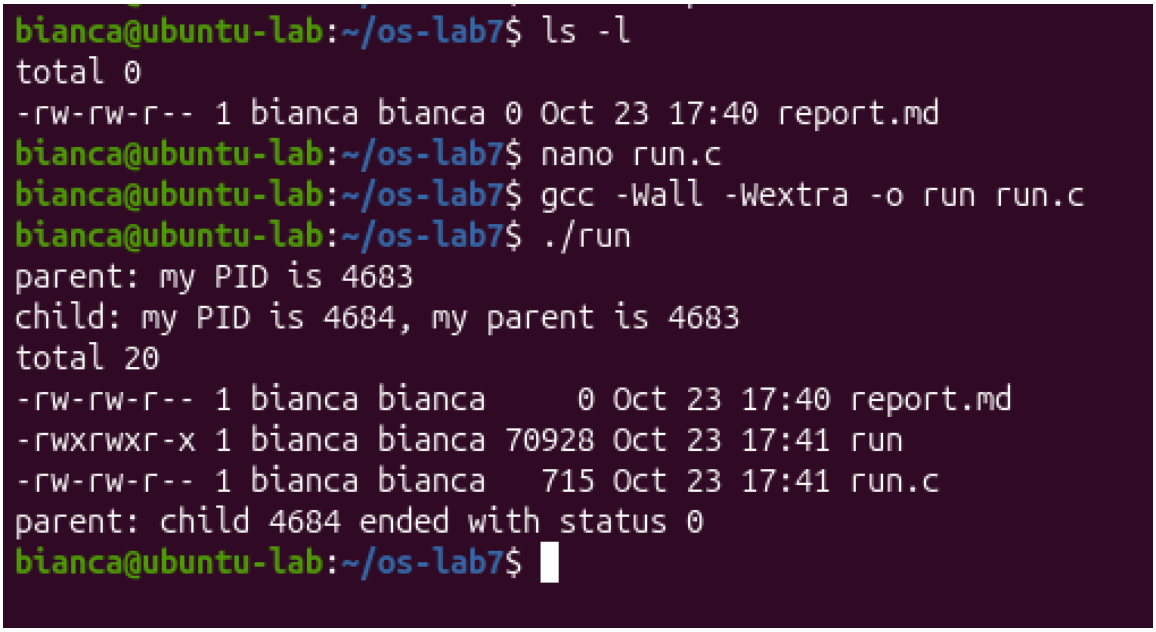


Figure 1: Creating a child process and executing ls -l

3.2 Replacing ls with date

The line invoking `ls` was replaced by:

```
execlp("date", "date", (char *)NULL);
```

The program was compiled again:

```
gcc -Wall -Wextra -o run run.c && ./run
```

Its output included:

```
parent: my PID is 4734
child: my PID is 4735, my parent is 4734
```

```
Fri Oct 23 05:43:47 PM UTC 2026
parent: child 4735 ended with status 0
```

The child ran `date` instead of `ls`, and the parent collected a successful exit status.

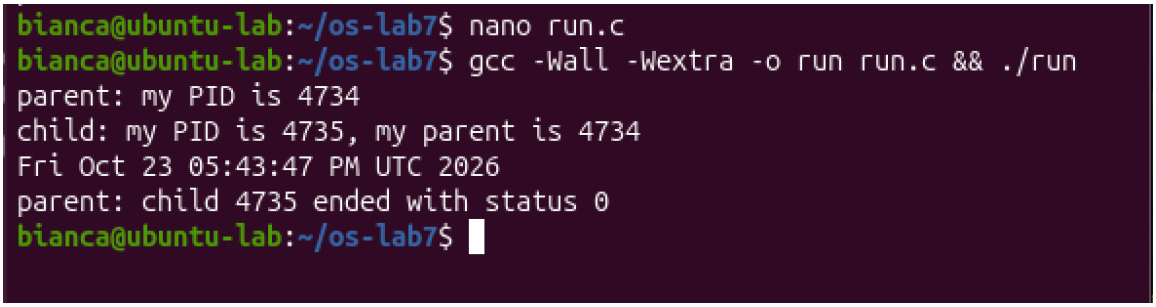


Figure 2: Executing date in the child process

3.3 Testing an Execution Failure

Next, a nonexistent executable was requested:

```
execlp("nosuchprog", "nosuchprog", (char *)NULL);
```

After recompilation and execution, the output was:

```
parent: my PID is 4842
child: my PID is 4843, my parent is 4842
exec: No such file or directory
parent: child 4843 ended with status 127
```

Because `execlp()` failed, control returned to the child's C code. The following `perror("exec")` printed the failure, and `exit(127)` supplied the status seen by the parent.

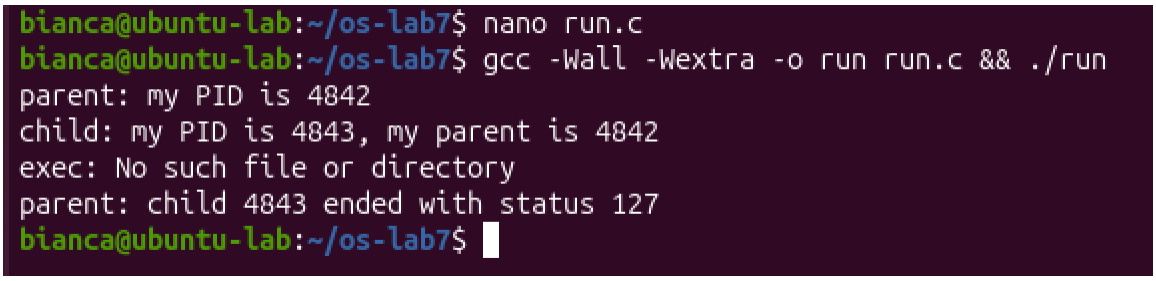


Figure 3: Failure to execute a nonexistent program

3.4 Part 1 – Observations

1. `perror("exec")` is printed only for `nosuchprog` because successful `execlp()` replaces the process image and does not return.

2. The number printed as "my parent" is the parent process's PID returned by `getppid()`. For the first run, it was `4683`.

3. The parent observed status `0` after `ls` and status `127` after `nosuchprog`.

4 Part 2 – A Zombie, on Purpose

4.1 Creating the zombie Program

A zombie is a terminated child whose exit status has not yet been collected by its parent. The original `run.c`, restored to execute `ls`, was copied to `zombie.c`:

```
cp run.c zombie.c
nano zombie.c
```

A delay was inserted immediately before waiting:

```
int status;
sleep(20);
waitpid(pid, &status, 0);
```

This gives the child time to finish while the parent still exists but has not yet collected its status.

4.2 Observing the Zombie in the Process Table

The experiment was started in the background, then the process table was inspected:

```
gcc -Wall -Wextra -o zombie zombie.c
./zombie > /dev/null &
sleep 2; ps -o pid,ppid,stat,cmd
```

The relevant output was:

```
PID PPID STAT CMD
3515 3452 Ss /usr/bin/bash
5222 3515 S ./zombie
5223 5222 Z [ls] <defunct>
5226 3515 R+ ps -o pid,ppid,stat,cmd
```

The parent `5222` was sleeping (`S`). The child `5223` appeared with status `Z`, meaning zombie.

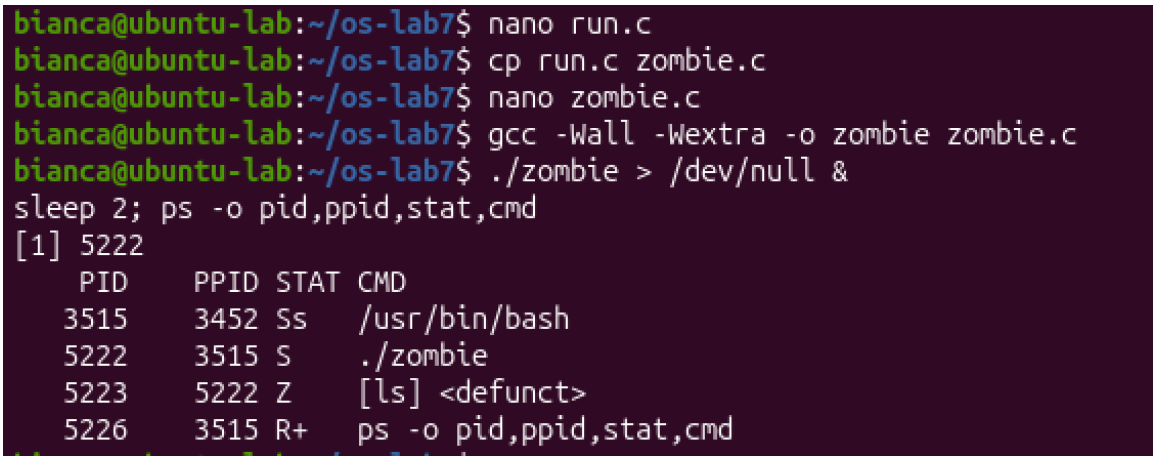


Figure 4: Zombie child shown with status Z and the defunct label

4.3 Inspecting the Process Table After the Delay

After the parent completed its sleep, the same process list was checked:

```
ps -o pid,ppid,stat,cmd
```

The output was:

```
PID PPID STAT CMD
3515 3452 Ss /usr/bin/bash
5238 3515 R+ ps -o pid,ppid,stat,cmd
[1]+ Done ./zombie > /dev/null
```

Neither `5222` nor `5223` remained. The parent called `waitpid()` to reap its terminated child and then exited normally.

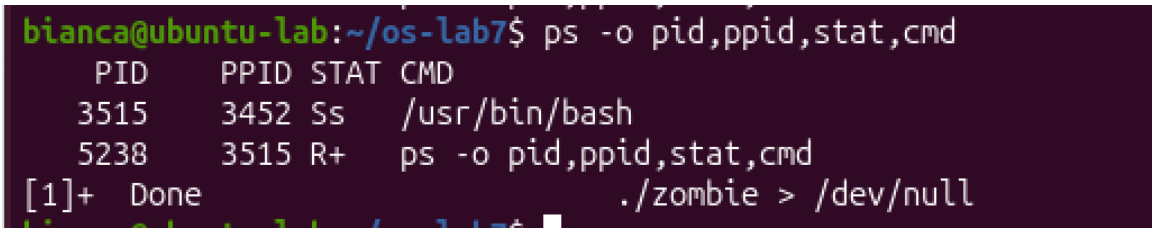


Figure 5: Process table after the zombie has disappeared

4.4 Part 2 – Observations

1. The `ls` child had `Z` in the `STAT` column. This indicates a terminated child waiting for its parent to collect its exit status.

2. After 20 seconds, the `[ls] <defunct>` entry had disappeared. The line `waitpid(pid, &status, 0);` allowed the parent to collect the child's termination status.

5 Part 3 – Redirection: ls -l to out.txt

5.1 Understanding Standard File Descriptors

Linux processes normally use three standard descriptors:

Descriptor	Meaning
0	Standard input (stdin)
1	Standard output (stdout)
2	Standard error (stderr)

The goal was to reproduce the shell operation `ls -l > out.txt` using C system calls.

5.2 Creating and Testing redirect.c

The program was created and the child's standard output was reassigned:

```
nano redirect.c
```

```
close(1);
int fd = open("out.txt",
    O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd < 0) {
    perror("open");
    exit(1);
}
fprintf(stderr, "open gave descriptor %d\n", fd);
execlp("ls", "ls", "-l", (char *)NULL);
```

Closing descriptor 1 first made it the lowest available number. When `open()` created or truncated `out.txt`, the system assigned descriptor 1 to it. The subsequent `ls` output therefore went into the file without modifying the `ls` program itself.

Commands executed:

```
gcc -Wall -Wextra -o redirect redirect.c && ./redirect
head -3 out.txt
```

Observed output:

```
open gave descriptor 1
done: the listing is in out.txt
total 60
-rw-r--r-- 1 bianca bianca 0 Oct 23 17:55 out.txt
-rwxrwxr-x 1 bianca bianca 70824 Oct 23 17:55 redirect
```

The `fprintf(stderr, ...)` message appeared on the terminal because descriptor 2 was not redirected.

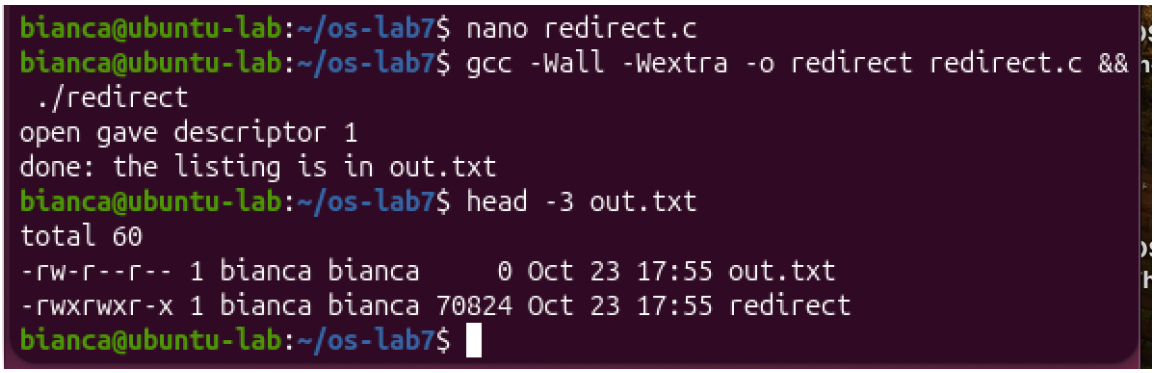


Figure 6: Writing the directory listing to out.txt

5.3 Part 3 – Observations

1. `open()` returned descriptor `1` because `close(1)` freed the lowest available file descriptor.

2. The listing did not appear on screen because `ls` wrote to its standard output, which now referred to `out.txt`.

3. To redirect errors as in `2> err.txt`, descriptor `2` would be closed before opening the target file.

6 Part 4 – A Pipe: ls /etc to wc -l

6.1 Creating pipeline.c

The final program implemented the shell pipeline `ls /etc | wc -l` using one pipe and two children. The call `pipe(p)` creates a reading end `p[0]` and a writing end `p[1]`.

The source file was created using:

```
nano pipeline.c
```

The first child made descriptor 1 point to the pipe writer:

```
close(1);
dup(p[1]);
close(p[0]);
close(p[1]);
execlp("ls", "ls", "/etc", (char *)NULL);
```

The second child made descriptor 0 point to the pipe reader:

```
close(0);
dup(p[0]);
close(p[0]);
close(p[1]);
execlp("wc", "wc", "-l", (char *)NULL);
```

The parent closed both of its own pipe endpoints before waiting for the children:

```
close(p[0]);
close(p[1]);
wait(NULL);
wait(NULL);
```

6.2 Comparing the C Pipeline with the Shell

The program was compiled and tested:

```
gcc -Wall -Wextra -o pipeline pipeline.c
./pipeline
ls /etc | wc -l
```

The two outputs were identical:

```
./pipeline
237
ls /etc | wc -l
237
```

This confirmed that the first child sent the directory listing through the pipe and the second child counted the received lines correctly.

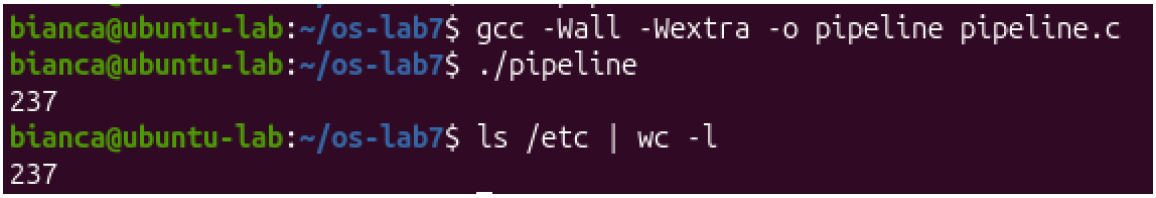


Figure 7: Matching line counts from the C and Bash pipelines

6.3 Leaving the Writing End Open

The laboratory required an intentional mistake. Only the parent's `close(p[1])` instruction was commented out:

```
close(p[0]);
/* close(p[1]); */
wait(NULL);
wait(NULL);
```

The program was rebuilt and run:

```
gcc -Wall -Wextra -o pipeline pipeline.c
./pipeline
```

This time it did not finish normally, and the terminal showed that it was interrupted using `Ctrl+C`. The `ls` child had finished, but the `wc` child still waited for end-of-file (EOF). Since the parent retained a writing endpoint, the pipe could not report EOF.

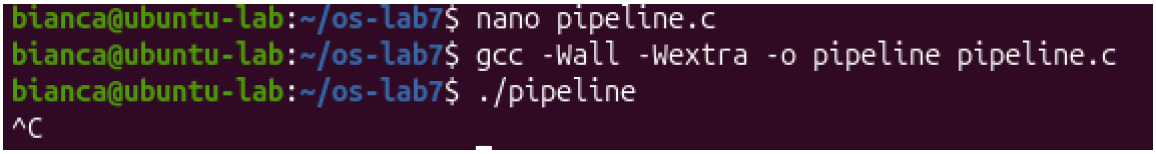


Figure 8: Pipeline blocked until interrupted with Ctrl+C

6.4 Restoring the Working Pipeline

The original statement `close(p[1]);` was restored in the parent, and the code was compiled again:

```
gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
```

The program once again printed `237` and returned to the shell prompt normally.

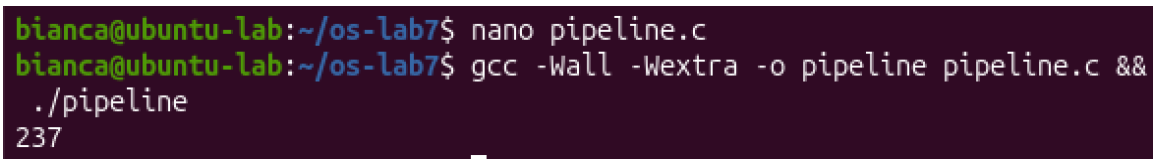


Figure 9: Correct pipeline restored and executed successfully

6.5 Part 4 – Observations

1. Yes. Both `./pipeline` and the shell command `ls /etc | wc -l` printed `237`.
2. When the parent's `close(p[1])` was commented out, `wc` was the child that did not end.
3. `wc` was waiting for EOF, which could not arrive because the parent still had the writing end open.

7 Results and Discussion

All four mandatory parts were completed successfully. The `run.c` experiment distinguished the parent and child PIDs and showed exit statuses `0` and `127` for successful and failed execution. The zombie experiment showed `[ls] <defunct>` with status `Z`, followed by its removal after the parent's `waitpid()` call.

The redirection experiment confirmed that closing and reopening descriptor 1 redirected `ls` output to `out.txt`. Finally, the pipeline experiment produced the same line count as the Bash pipeline: `237`. The intentionally unclosed writing end demonstrated why all unused pipe descriptors must be closed.

8 GitHub Repository

The required laboratory files were stored in:

```
/home/bianca/os-lab7
```

The repository includes:

```
run.c
zombie.c
redirect.c
pipeline.c
report.md
```

The local repository was initialized and its files committed:

```
git init
git branch -M main
git add run.c zombie.c redirect.c pipeline.c report.md
git commit -m "Complete Operating Systems Lab 7"
```

Git reported commit `bb45829`. The remote repository was configured and pushed using:

```
git remote add origin https://github.com/Biancatataroi/Operating-Systems-Lab7.
    git
git push -u origin main
```

The terminal confirmed:

```
[new branch] main -> main
branch 'main' set up to track 'origin/main'.
```

The repository is available at:

https://github.com/Biancatataroi/Operating-Systems-Lab7

9 Conclusion

In this laboratory work, I investigated the Linux process-management and interprocess-communication mechanisms through practical C programs.

In the first part, I created a child process using `fork()`, ran commands using `execlp()`, and collected exit status with `waitpid()`. Changing the child command from `ls` to `date` showed that the same process-management structure could launch different programs. The failure case showed how `perror()` reports an error and how a specific exit code can be returned to the parent.

In the second part, I observed a zombie process by inserting a delay before `waitpid()`. The process table showed the terminated child with status `Z`, and the entry disappeared after its parent collected the exit status. This showed why waiting for children matters.

In the third part, I used file descriptors to redirect the standard output of `ls` into `out.txt`. This experiment clarified the relationship between standard output and the file descriptors managed by Linux.

Finally, I created a pipe connecting two child processes. The C implementation and the equivalent Bash pipeline both counted 237 lines. Intentionally leaving the writing end open caused the receiver to wait indefinitely for EOF; restoring the close operation allowed normal termination.

Overall, this laboratory improved my practical understanding of process creation, execution, waiting, zombie states, redirection, pipes, and the importance of closing unused file descriptors.